

OVERVIEW

 Agenda

🕒 1 min

Logistic Regression Intuition

- Approach, Hyperplanes, Thresholding
- Code implementation of the sigmoid function
- Why is it called "regression" if we are doing classification?

Comparing y vs $\hat{y}$:

- Log-loss intuition, combining both cases into one equation, implementing log-loss

Optimization

- Regularisation
- Performance — Accuracy Metric

Scikit-Learn Implementation of Logistic Regression

- EDA, Train/Test Split, Regularization implementation
- `predict_proba` function from sklearn, sklearn pipeline

Impact of outliers

REVIEW

 Recap, Problem Setup & Data

🕒 5 min

In the previous module, we covered:

- Introduction to Supervised Learning
- Linear Regression & Gradient Descent
- Multiple Linear Regression
- Model Evaluation (R^2 & Adjusted R^2)
- Assumptions of Linear Regression
- Bias-Variance Tradeoff
- Regularization (Ridge, Lasso & Elastic Net)
- Cross Validation

Key Takeaways

- Supervised learning uses labeled data to learn patterns.
- Regression is used when the target variable is continuous.
- Building a good ML model involves preprocessing, training, evaluation, and validation.
- Techniques like regularization and cross-validation help improve model generalization.

TODAY'S FOCUS

So far, we've learned how to predict continuous values. Now we shift focus to **Classification**, where the goal is to predict discrete categories such as Spam/Not Spam, Fraud/Not Fraud, or Disease/No Disease.

Problem Statement — Telecom Churn Analysis

Churn (loss of customers to competition) is an expensive problem for telecom companies, since acquiring a new customer costs far more than retaining an existing one.

Existing customers typically consume more services and generate additional referrals, which is why businesses invest heavily in measuring and predicting churn.

Business Objective

Customer retention is best achieved by truly knowing the customer. The vast volumes of data collected can be used to build churn prediction models, so the company can prioritize focused marketing on the customers most likely to defect.

As a Data Analyst, the task is to build ML algorithms that predict and automate churn detection.

Data Description

Records: 5700 · **Features:** 21

Feature	Description
state	2-letter code of the US state of customer residence
account_length	Number of months the customer has been with the current telco provider
area_code	string="area_code_AAA" where AAA = 3 digit area code
intl_plan	The customer has international plan
vmail_plan	The customer has voice mail plan
vmail_messages	Number of voice-mail messages
day_mins / day_calls / day_charge	Total minutes, count and charge of day calls

eve_mins / eve_calls / eve_charge	Total minutes, count and charge of evening calls
night_mins / night_calls / night_charge	Total minutes, count and charge of night calls
intl_mins / intl_calls / intl_charge	Total minutes, count and charge of international calls
customer_service_calls	Number of calls to customer service
phone	10 digit number
churn	Customer churn — target variable

Initial Setup

```
# Necessary Imports
import numpy as np
import matplotlib.pyplot as plt
```

```
# Downloading the Dataset
!gdown 1uUt7uL-VuF_5cpodYRiriEwhsldeEp3m
```

OUTPUT

```
Downloading...
From: https://drive.google.com/uc?id=1uUt7uL-VuF_5cpodYRiriEwhsldeEp3m
To: /content/churn_logistic.csv
100% 494k/494k [00:00<00:00, 28.0MB/s]
```

```
import pandas as pd
churn = pd.read_csv("churn_logistic.csv")
churn.head()
```

The first rows show columns like `Account Length` , `Day Mins` , `CustServ Calls` , `Intl Plan` , `State` , and the target `Churn` (0/1).



CONCEPT

Logistic Regression Intuition

🕒 4 min

Linear Regression, Revisited

In linear regression we fit a line or plane to most of the points, then use that line to predict values for future inputs — like predicting the price of a car from its age. That plot looked like a good fit.

Now the Problem Changes

We want to identify **classes**, not continuous values. We have data points for two classes: class 1 (dot, means churned) and class 2 (cross, means not churned).

Given some new, unlabeled data points, how do we identify which class they belong to?

Approach

- Observe the target variable: two different classes → this is a case of **classification**.
- The simplest approach is to draw a **line** that separates the datapoints between the two classes.
- Points on one side of the line are class 0; points on the other side are class 1.

We want a line (or plane) such that points fall on either side of it. This gives us the **hyperplane**:

$$z = w_0 + w_1x_1 + w_2x_2 + \dots + w_dx_d$$

You can also check how this plane separates the classes in 3D.



MATH

Hyperplanes

🕒 4 min

A hyperplane is an n -dimensional generalization of a plane — an affine subspace of dimension $n - 1$ that splits an n -dimensional space.

In 3D space, a hyperplane is one dimension less than 3, i.e. a 2D plane. In 2D space, one dimension less is a single-dimensional line, and so on. Hyperplanes are the general entity — line, plane, or higher-dimensional separator depending on the number of dimensions.

Distance Implies Confidence

If we consider the line/plane as the separator, which points are more probable of being in class 1? The points that are **farther away** from the separator are more probable of being in that class.

So, the more the distance from the separator, the more the probability of belonging to that class.



FROM DISTANCE TO PROBABILITY

A hyperplane gives us a distance score, telling us how far a point lies from the decision boundary. But distance alone is hard to interpret in business terms. Saying "there is an 85% chance this

customer will churn" is far more useful than "distance = 4.2" — that's exactly why we need a function that converts raw scores into probabilities.

Can we use the linear function directly for predictions? Note that the true values y_i are $\{0, 1\}$, but our linear function z ranges over $\{-\infty$ to $+\infty\}$.

We need a filter/function to compress that range into 0–1. That function is the **sigmoid function** — also called the **logistic function**.



QUIZ 1

Distance from the Decision Boundary

⌚ 1 min

The farther a point is from the line or hyperplane, the lower the chances that it belongs to a specific class.

☐ True☐ False EXPLANATION

It's the opposite. Distance and confidence are directly proportional — the farther a point lies from the hyperplane, the **higher** the probability that it belongs to that class, not lower.



CONCEPT

Sigmoid Function & Thresholding

⌚ 6 min

We take the input $z = w_0 + w_1x_1 + w_2x_2 + \dots + w_dx_d$ and pass it through the sigmoid (logistic) function, so the output lies between 0 and 1.



INTERACTIVE VISUALIZER

See how the sigmoid function compresses any input value:

[Sigmoid Function — Input/Output Transformation →](#)

The sigmoid function is probabilistic in nature — its output between 0 and 1 can be interpreted as the probability of belonging to class 1.

- Sigmoid output of **0.7** → 70% probability of being in class 1
- Sigmoid output of **0.2** → 20% probability of class 1, i.e. 80% probability of class 0

🚩 FROM PROBABILITY TO DECISION

The model gives us probabilities, which is useful. But most business problems eventually need a hard yes/no decision — so we need a rule to convert probabilities into final class labels.

Thresholding

The actual labels y_i are exactly 0 or 1. But the predicted value $\hat{y}_i$ is somewhere between 0 and 1 — it may be 0.7, 0.2, or 0.34.

So we set a **threshold**, above which we assign one class and below which we assign the other.

The threshold is not always 0.5 — it can depend on the use case.



IMPLEMENTATION

Code Implementation of the Sigmoid Function

🕒 4 min

```
import math # A library that manages mathematics
```

```
def sigmoid(x):  
    return 1/(1+np.e**-x)
```

```
x = np.linspace(-10, 10, 20) # 20 equidistant points between -10 and 10  
z = sigmoid(x)  
  
plt.plot(x, z)  
plt.xlabel("x")  
plt.ylabel("Sigmoid(X)")  
plt.grid()  
plt.show();
```

Output Plot Analysis

- When $\sigma(x)$ is close to 0.5, the rate at which the sigmoid value increases/decreases is very fast.
- As $\sigma(x)$ moves further away from 0.5 toward 1 or 0, the rate of change plateaus.



CONCEPT

Why Is It Called "Logistic Regression" if We Are Doing Classification?

⌚ 2 min

The term "logistic regression" comes from the logistic (sigmoid) function.

- At its base, it's doing **regression** only — we output a real value between 0 and 1 that we treat as a probability.
- On top of that regression, we apply the **logistic function** — hence, logistic regression.
- The true labels y_i are 0 or 1, but our predicted $\hat{y}_i$ ranges continuously between 0 and 1.



QUIZ 2

Interpreting the Sigmoid Output

⌚ 1 min

In logistic regression, the output of the sigmoid function is interpreted as:

- ☐ Class probabilities
- ☐ Raw scores
- ☐ Error rates
- ☐ Regression coefficients

 EXPLANATION

The sigmoid squashes the linear score z into $[0, 1]$, and this value is interpreted as the probability of the input belonging to class 1.



CONCEPT

Comparing y_i vs $\hat{y}_i$

⌚ 3 min



BEYOND PREDICTING PROBABILITIES

Predicting probabilities is only half of model training. The next question is: how does the model know whether its predictions are good or bad during training?

We use **Log-Loss** as the loss function for classification.

 REFERENCE

Log-loss comes from the principle of Maximum Likelihood Estimation. We won't derive it from probability theory here — instead we'll build intuition for why it makes sense and how it behaves.



MATH

Log-Loss Intuition

🕒 5 min

Let's plot a graph between error and $\hat{y}$.

Case: $y_i = 1$

- If $\hat{y}_i = 1 \rightarrow$ error is very low, near 0.
- If $\hat{y}_i = 0 \rightarrow$ error is very high (say, infinity).
- If $\hat{y}_i = 0.99 \rightarrow$ error is slightly greater than when $\hat{y}_i$ was 1.
- If $\hat{y}_i = 0.01 \rightarrow$ error is slightly lesser than infinity.

This is exactly the plot of the equation:

$$-\log(\hat{y})$$

We don't worry about the part of the curve below $\hat{y} = 0$, since $\hat{y}_i$ is always between 0 and 1 — so the loss is always positive.

Case: $y_i = 0$

- If $\hat{y}_i = 0 \rightarrow$ error ≈ 0 .
- If $\hat{y}_i = 1 \rightarrow$ error $= \infty$.

This curve is the plot of the equation:

$$-\log(1 - \hat{y})$$

So mathematically, we can define log-loss using these two cases:

 **ONE EQUATION, NOT TWO**

We now have two separate loss equations — one for when the actual class is 1, another for when it's 0. For optimization, working with a single unified equation is much more convenient.



MATH

Combining Both Cases into One Equation

⌚ 3 min

Using a small piece of clever math, we can combine both log-loss cases by simply adding them — one of the two terms always evaluates to zero for a given sample:

$$L = -y \cdot \log(\hat{y}) - (1-y) \cdot \log(1-\hat{y})$$

When $y = 1$, the second term vanishes and we're left with $-\log(\hat{y})$. When $y = 0$, the first term vanishes and we're left with $-\log(1-\hat{y})$ — exactly the two cases we derived earlier.



IMPLEMENTATION

Implement Log-Loss

⌚ 5 min

Case 1: $y_i = 1, \hat{y}_i = 0.9$

```
import math

y = 1
yhat = 0.9
print(-y*math.log(yhat)-(1-y)*math.log(1-yhat))
```

OUTPUT

```
0.10536051565782628
```

The loss is almost 0.1 — close to 0. Let's bring $\hat{y}_i$ even closer to 1.

Case 2: $y_i = 1, \hat{y}_i = 0.99$

```
y = 1
yhat = 0.99
print(-y*math.log(yhat)-(1-y)*math.log(1-yhat))
```

OUTPUT

```
0.01005033585350145
```

Just incrementing $\hat{y}_i$ by 0.09 causes a drastic decrease in loss. Now let's see what happens as $\hat{y}_i$ moves closer to 0.

Case 3: $y_i = 1, \hat{y}_i = 0.1$

```
y = 1
yhat = 0.1
print(-y*math.log(yhat)-(1-y)*math.log(1-yhat))
```

OUTPUT

```
2.3025850929940455
```

Notice how much higher the log-loss is. Let's bring $\hat{y}_i$ even closer to 0.

Case 4: $y_i = 1, \hat{y}_i = 0.01$

```
y = 1
yhat = 0.01
print(-y*math.log(yhat)-(1-y)*math.log(1-yhat))
```

OUTPUT

```
4.605170185988091
```

A small variation in $\hat{y}_i$ causes a drastic increase in loss. Now let's look at the scenario when the actual label is 0.

Case 5: $y_i = 0, \hat{y}_i = 0.1$

```
y = 0
yhat = 0.1
print(-y*math.log(yhat)-(1-y)*math.log(1-yhat))
```

OUTPUT

```
0.10536051565782628
```

A small loss, identical to Case 1. Now let's bring $\hat{y}_i$ closer to 0.

Case 6: $y_i = 0, \hat{y}_i = 0.01$

```
y = 0
yhat = 0.01
```

```
print(-y*math.log(yhat)-(1-y)*math.log(1-yhat))
```

OUTPUT

```
0.01005033585350145
```

Loss decreased, similar to Case 2.

Case 7: $y_i = 0$, $\hat{y}_i = 0.9$

```
y = 0
yhat = 0.9
print(-y*math.log(yhat)-(1-y)*math.log(1-yhat))
```

OUTPUT

```
2.302585092994046
```

As expected, the loss is high.

Summary

- Log-loss **decreases** as $\hat{y}_i$ gets closer to y_i .
- Log-loss **increases** as $\hat{y}_i$ gets farther from y_i .
- This means we can use optimization techniques, like gradient descent, on the log-loss function to find the optimal weights of the logistic regression model.



QUIZ 3

Log-Loss for a Confident, Correct Prediction

⌚ 1 min

Suppose the actual label is $y = 1$ and the predicted probability is $\hat{y} = 0.8$. What will the log-loss be?

☐ Log-loss will be a very high value.

☐ Log-loss will be a very low value.

☐ Log-loss will be 0.



EXPLANATION

Since $\hat{y} = 0.8$ is fairly close to the actual label $y = 1$, the $-\log(\hat{y})$ term stays small. The closer $\hat{y}$ gets to y , the closer the log-loss gets to 0 — but it never reaches exactly 0 unless $\hat{y} = 1$.



CONCEPT

Optimization

⌚ 3 min

So far we've only seen log-loss for the i -th sample. We define the full loss function for logistic regression as the sum of log-loss over all n samples:

$$L_{all} = \sum_i [-y \cdot \log(\hat{y}_i) - (1-y) \cdot \log(1-\hat{y}_i)]$$

We use gradient descent to optimize this equation.

🚀 BETTER PREDICTIONS → LOWER LOSS

Minimizing loss helps us fit the training data well — but fitting too well can become a problem of its own. This is where regularization, which we saw in linear regression, comes back into play.



CONCEPT

Regularisation

⌚ 4 min

Similar to regression, logistic regression can also use a regularizer. We optimize the weights (w_j) using the same regularization parameter λ .

🚀 IN CODE

Sklearn takes a parameter `C`, which is the **inverse** of λ .

🤔 ASK YOURSELF

If there is no non-linear feature, should we still use regularization when performing min L?

Yes — to avoid overfitting the model.

We add an L2 regularization term to the loss function L . Gradient descent now performs gradients on:

$$\min \sum_j [L + \lambda \sum_j w_j^2]$$

**INTERACTIVE VISUALIZER**

See how regularization affects the decision boundary:

[Regularization — Effect on Decision Boundary →](#)

**QUIZ 4****Choosing a Threshold for Cancer Detection**

⌚ 1 min

You are building a model to predict whether a person has cancer. Your goal is to avoid missing any cancer patient, even if it means identifying some healthy people as cancer-positive. Which threshold should you choose?

- ☐ If predicted probability > 0.5, classify as having cancer.
- ☐ If predicted probability > 0.7, classify as having cancer.
- ☐ If predicted probability > 0.9, classify as having cancer.
- ☐ If predicted probability > 0.3, classify as having cancer.

**EXPLANATION**

Lowering the threshold makes it "easier" for a data point to be classified as cancer-positive. This increases recall — fewer actual cancer patients slip through as false negatives — at the cost of flagging more healthy people as false positives, which is an acceptable tradeoff here.

**METRIC****Performance Metric — Accuracy**

⌚ 4 min

**FROM TRAINING TO EVALUATION**

So far, we've focused on training the model correctly. Once training is complete, the next question is: how do we evaluate whether the model is actually performing well?

Accuracy is calculated by dividing the number of correct predictions by the total number of samples.

Given data where y is the correct label, $\hat{y}$ is the predicted output, and `y-pred` is the model's raw scalar output (before sigmoid) — entries where y and $\hat{y}$ differ are called **misclassifications**.

🧐 SUPPOSE 100 DATA POINTS, 10 MISCLASSIFIED

How many are accurate? 90.

What is the accuracy? $90/100 = \text{correct predictions} / \text{total predictions}$.

$$\text{Accuracy} = \text{Correct Predictions} / \text{Total Predictions}$$



QUIZ 5

Calculating Classification Accuracy

🕒 1 min

Suppose you have 1500 samples, with 750 samples in Class 1 and 750 samples in Class 2. A logistic regression model correctly predicts 450 samples from Class 1 and 350 samples from Class 2. What is the accuracy of the model?

☐ 0.53

☐ 1.06

☐ 0.60

☐ 0.46

💡 EXPLANATION

Accuracy = $(450 + 350) / 1500 = 800 / 1500 \approx 0.53$.



IMPLEMENTATION

Scikit-Learn Implementation of Logistic Regression

🕒 3 min

 FROM THEORY TO CODE

We've now covered the theory behind logistic regression — from intuition to optimization and evaluation. Let's move into implementation and build this model on a real dataset using sklearn.

```
# Churn prediction in telecom.
import numpy as np
import matplotlib.pyplot as plt
```

```
# Downloading the file
!gdown 1uUt7uL-VuF_5cpodYRiriEwhsldeEp3m
```

OUTPUT

```
Downloading...
From: https://drive.google.com/uc?id=1uUt7uL-VuF_5cpodYRiriEwhsldeEp3m
To: /content/churn_logistic.csv
100% 494k/494k [00:00<00:00, 53.6MB/s]
```

```
import pandas as pd
churn = pd.read_csv("churn_logistic.csv")
churn.head()
```

Let's see how many samples the churn data has:

```
churn.info()
```

OUTPUT

```
<class 'pandas.core.frame.DataFrame'>
RangeIndex: 5700 entries, 0 to 5699
Data columns (total 21 columns):
 #   Column                Non-Null Count  Dtype  
---  -
 0   Account Length       5700 non-null   int64  
 1   VMail Message        5700 non-null   int64  
 2   Day Mins              5700 non-null   float64
...
19  Phone                5700 non-null   object  
20  Churn                5700 non-null   int64  
dtypes: float64(8), int64(11), object(2)
memory usage: 935.3+ KB
```

5700 samples — quite enough for training the logistic regression model.

```
churn["Churn"].value_counts()
```

Churn	Count
0	2850
1	2850

The data is **balanced** — equal samples for customers who churned and who did not.



IMPLEMENTATION

Exploratory Data Analysis (EDA)

🕒 5 min

Let's check how each feature relates to churn with a quick univariate EDA.

```
import seaborn as sns
sns.boxplot(x='Churn', y='Day Mins', data = churn)
```

🤔 WHAT CAN WE SEE FROM THE BOXPLOT?

Churned customers have a higher median **Day Mins** than non-churned customers — customers who spend more time on day calls tend to churn. This shows **Day Mins** is an important feature.

Now let's check for collinearity with **Day Mins**:

```
sns.pairplot(data=churn, y_vars=["Day Mins"], x_vars=['Account Length', 'VMail Message',
    'Intl Mins', 'CustServ Calls', 'Churn', 'Intl Plan', 'VMail Plan',
    'Day Calls', 'Day Charge', 'Eve Calls', 'Eve Charge', 'Night Calls',
    'Night Charge', 'Intl Calls', 'Intl Charge'], height=4, aspect=1)
plt.show()
```

🤔 WHICH FEATURE IS HIGHLY CORRELATED WITH DAY MINS?

Day Charge is highly correlated — as **Day Mins** increases, so does **Day Charge**. We drop one of them. Notice we're doing the same feature-selection steps as in linear regression, since logistic regression is still training a linear model in log-odds space.

Now let's check **Account Length**:

```
sns.boxplot(x = 'Churn', y= 'Account Length', data = churn)
```

🤔 IS ACCOUNT LENGTH RELEVANT?

The medians for both classes are quite similar — this feature has no real significance and can be dropped.

What EDA Can Do

- Remove useless features
- Remove highly correlated features

We select a few features (not necessarily the best) to move forward with:

```
cols = ['Day Mins', 'Eve Mins', 'CustServ Calls', 'Intl Plan', 'VMail Message']
y = churn["Churn"]
X = churn[cols]
X.shape
```

OUTPUT

```
(5700, 5)
```



IMPLEMENTATION

Train Test Split

⌚ 4 min

We split the data as done in linear regression: **80%** training, **20%** test. Since we know the importance of validation data, we split the 80% further into **60% training** and **20% cross-validation**.

```
from sklearn.model_selection import train_test_split

X_tr_cv, X_test, y_tr_cv, y_test = train_test_split(X, y, test_size=0.2, random_state=4)
X_train, X_val, y_train, y_val = train_test_split(X_tr_cv, y_tr_cv, test_size=0.25, random_state=4)
X_train.shape
```

OUTPUT

```
(3420, 5)
```

Standardization

The scaler is fit **only** on the training data, then used to transform train, validation and test data.

```
from sklearn.preprocessing import StandardScaler
scaler = StandardScaler()
scaler.fit(X_train)

X_train = scaler.transform(X_train)
X_val = scaler.transform(X_val)
X_test = scaler.transform(X_test)
```

```
# print X_train
X_train
```

OUTPUT

```
array([[ -1.3401872 , -2.29116555, -0.52179332,  2.19011647, -0.5352414 ],
       [ -0.37273287, -1.19169576, -1.15081453, -0.45659672, -0.5352414 ],
       [  1.54289097,  0.33984637, -0.52179332, -0.45659672, -0.5352414 ],
       ...,
       [  0.83417443,  0.52694913, -1.15081453, -0.45659672, -0.5352414 ],
       [ -0.58807985, -0.12887496,  1.3652703 , -0.45659672, -0.5352414 ],
       [ -0.45469329,  1.42195611,  0.73624909,  2.19011647,  1.68255781]])
```



IMPLEMENTATION

Regularization Implementation

⌚ 5 min

LogisticRegression Using Scikit-Learn

- L2 regularization is applied by **default**.
- Sklearn exposes `C`, the **inverse** of the regularization parameter λ , instead of λ directly.



ELASTICNET'S L1_RATIO

Sklearn's `l1_ratio` hyperparameter ranges $0 \leq l1_ratio \leq 1$:

- `l1_ratio = 0` is equivalent to `penalty='l2'`
- `l1_ratio = 1` is equivalent to `penalty='l1'`
- $0 < l1_ratio < 1 \rightarrow$ combination of L1 and L2

```
from sklearn.linear_model import LogisticRegression
model = LogisticRegression() # C = 1, lambda = 1
model.fit(X_train, y_train)
```

Let's inspect the weights of each feature (excluding w_0 , the intercept):

```
model.coef_
```

OUTPUT

```
array([[ 0.78674301,  0.31084942,  0.94070165,  0.86919545, -0.31809071]])
```



WHAT CAN WE SAY ABOUT CUSTSERV CALLS?

Feature 3, `CustServ Calls`, has the highest weight. Since the data is standardized, this large positive value means customers with more customer-service calls are more likely to churn.

Next highest weight is `Intl Plan` — customers on an international plan churn more, suggesting an issue with the company's international plans.

```
model.intercept_
```

OUTPUT

```
array([0.05297365])
```

```
y_pred=model.predict(X_val)
print(y_pred)
```

OUTPUT

```
[0 0 1 ... 0 0 0]
```

We get the class labels for each input using `predict()`.

🔴 `PREDICT() VS PREDICT_PROBA()`

`predict()` gives only the final class labels. But logistic regression internally works with probabilities first, and those probabilities often provide deeper insight into model confidence.



IMPLEMENTATION

`predict_proba` Function from Sklearn

🕒 3 min

`predict_proba()` returns the class probabilities for each data point. It accepts the input data and returns an array of lists containing the class probabilities, ordered by class label.

```
y_pred_prob=model.predict_proba(X_val)
print(y_pred_prob)
```

OUTPUT

```
[[0.70985217 0.29014783]
 [0.8681417  0.1318583 ]
 [0.45621812 0.54378188]
 ...
 [0.95273524 0.04726476]
 [0.60128439 0.39871561]
 [0.81202669 0.18797331]]
```

Each row is the probability of that data point being in each class — and the probabilities in a row always add up to 1.

Checking accuracy score on train and validation data:

```
print(f'Train Accuracy:{model.score(X_train,y_train)}, Val Accuracy:{model.score(X_val,y_val)}')
```

OUTPUT

```
Train Accuracy:0.7564327485380117, Val Accuracy:0.7578947368421053
```

Both train and validation accuracy are quite low. Let's tune the hyperparameter $C = 1/\lambda$ to improve performance.



IMPLEMENTATION

Sklearn Pipeline

⌚ 8 min

A **Pipeline** chains multiple processing steps into a single scikit-learn estimator, with the same **fit**, **predict** and **score** methods as any other estimator.

```
from sklearn.pipeline import make_pipeline
train_scores = []
val_scores = []
scaler = StandardScaler()
for la in np.arange(0.01, 12000.0, 100): # range of values of Lambda
    scaled_lr = make_pipeline( scaler, LogisticRegression(C=1/la))
    scaled_lr.fit(X_train, y_train)
    train_score = scaled_lr.score(X_train, y_train)
    val_score = scaled_lr.score(X_val, y_val)
    train_scores.append(train_score)
    val_scores.append(val_score)
```

1. Initialize the range of values for λ .
2. For each value of λ , create a pipeline of standardization + logistic regression.
3. Fit the training data to the model.
4. Record the train and validation scores.

NOTE

The default scoring metric here is **Accuracy**.

```
len(val_scores)
```

OUTPUT

120

Let's plot the graph and pick the λ that gives the best validation score:

```
plt.figure()
plt.plot(list(np.arange(0.01, 12000.0, 100)), train_scores, label="train")
plt.plot(list(np.arange(0.01, 12000.0, 100)), val_scores, label="val")
plt.legend(loc='lower right')

plt.xlabel("Regularization Parameter( $\lambda$ )")
plt.ylabel("Accuracy")
plt.grid()
plt.show()
```

Validation accuracy rises to a peak and then decreases. As regularization increases, accuracy keeps decreasing since the model moves toward **underfitting**.

Picking the Best λ

We pick the λ where the validation score is maximum:

```
np.argmax(val_scores)
```

OUTPUT

```
np.int64(23)
```

```
val_scores[23]
```

OUTPUT

```
0.762280701754386
```

Validation score is maximum (76.2%) at index 23, so our best λ is:

$$l_best = 0.01 + 23 \times 100$$

(since we step by 100 in each iteration). Let's train the final model with this best λ :

```
scaled_lr = make_pipeline( scaler, LogisticRegression(C=1/l_best))
scaled_lr.fit(X_train, y_train)
```

Now let's test this best model on the held-out test data:

```
test_score = scaled_lr.score(X_test, y_test)
```

```
print(test_score)
```

OUTPUT

```
0.7684210526315789
```

 ON THE SCALER IN THE PIPELINE

The scaler performs scaling first, then logistic regression runs. It uses the training data's mean and standard deviation during fit, and continues using those same statistics for validation and test data — since we only call `.score()` on them, never `.fit()`.



QUIZ 6

Choosing the Best Regularization Parameter

 1 min

When using different values of λ , the model produces the following results. Which is the best value of λ ?

λ	Train Accuracy	Test Accuracy
1	0.90	0.78
0.1	0.98	0.99
0.01	0.93	0.97

☐ 1☐ 0.1☐ 0.01 EXPLANATION

Test accuracy is maximum when $\lambda = 0.1$, and the gap between train and test accuracy is very small — showing the model neither overfits nor underfits at this value. At $\lambda = 1$, both accuracies drop and the train-test gap widens, a sign of underfitting from too much regularization.



CONCEPT

Impact of Outliers

⌚ 3 min

Case I – Outlier on the Correct Side

Here $\hat{y} = \sigma(z_i)$, and since the point sits on the correct side of the boundary, the loss is very small — the impact of this outlier is **very low**.

Case II – Outlier on the Wrong Side

Say $z_i = -4.3$, so $\hat{y}$ becomes 0.01. Then:

$$L = -\log(0.01) \approx 4.6$$

This is a very large loss value — so the impact of this outlier on training is **very high**. It pulls the decision boundary toward itself in order to reduce that loss.

The best approach is to find such outliers and remove them, to get more accurate and stable results.



QUIZ 7

Effect of Outliers on the Decision Boundary

⌚ 1 min

How do outliers affect the classification boundaries in logistic regression?

- ☐ Outliers shift the classification boundaries closer to the outlier values.
- ☐ Outliers have no effect on the classification boundaries.
- ☐ Outliers widen the gap between the classification boundaries.
- ☐ Outliers make the classification boundaries more sensitive to minor changes.

💡 EXPLANATION

When an outlier lands on the wrong side of the boundary, its log-loss becomes very large (e.g. ≈ 4.6). During gradient descent, this large loss contributes a strong gradient that pulls the decision boundary toward that point to reduce its loss.



SUMMARY

Wrapping Up

⌚ 1 min

Everything we've seen so far assumes binary classification — only two classes. But many real-world problems involve more than two possible outcomes.

- ▶ Classification predicts discrete classes, not continuous values
- ▶ A hyperplane $z = w_0 + w_1x_1 + \dots$ separates classes
- ▶ Sigmoid squashes $z \in (-\infty, \infty)$ into a probability $\in (0, 1)$
- ▶ A threshold (not always 0.5) converts probability $\rightarrow$ class label
- ▶ Log-loss: $-y \cdot \log(\hat{y}) - (1-y) \cdot \log(1-\hat{y})$
- ▶ Log-loss is minimized via gradient descent
- ▶ L2 regularization (parameter $C = 1/\lambda$) prevents overfitting
- ▶ Accuracy = correct predictions / total predictions
- ▶ `predict_proba` exposes the underlying probabilities
- ▶ Sklearn pipelines chain scaling + model into one estimator
- ▶ Best λ maximizes validation accuracy while keeping train/test close
- ▶ Outliers on the wrong side of the boundary distort it — remove them

That is what we'll tackle in the upcoming lecture: multi-class classification, followed by evaluating classification models with accuracy, precision, recall, and more.